论文 × 代码 · 中文可视化阅读版

Cascade Routing 代码框架是怎样运转的?

从整体框架，到公式含义，再到一次完整的模型选择。

本页可以离线阅读。建议先读前三部分，再按目录进入完整报告。

查看原始 Markdown

来源：/home/guoqiuyuan/LLMresearch/cascade-routing/reports/framework_reading_20261012_01/FRAMEWORK_GUIDE.md
SHA256：36f287287c0ce73759c0d223377d8257a1b56b52652a25346836ebca37131532
生成时间：2026-10-12T12:34:29.984025+00:00 · 内容依据 2026-10-12 阅读报告

先建立整体印象

四个角色，一条闭环

可以把系统理解为：先预测，再做决定，再执行，最后评价。原论文主要解决前两个环节；本地工程还管理执行约束、资源和事后评分。

01

估计器

“每个模型可能答得多好？要花多少资源？”

`quality / cost computer`

02

选择算法

“下一步调用谁？还是现在停止？”

`CascadeRouter`

03

执行器

检查动作，发起调用，保留时间和费用。

`UnifiedExecutor`

04

终局评估

答案固定之后，再用独立标签判断是否达标。

`evaluate_terminal`

> 控制器可以读取已经返回的回答；不能读取真实质量标签、未调用模型的答案或未来资源。这里的“预测质量”与事后评分是两种信息。

公式翻译成中文

先记住一句话：质量收益 − 成本惩罚

方案得分 = 预测质量 − 成本重视程度 × 预测成本

$$\tau_i(x) = \hat{q}_i(x) - \lambda \cdot \hat{c}_i(x)$$

符号	怎么读	实际含义
x / i	问题 / 模型编号	正在处理哪个问题、考虑哪个模型
$\hat{q}$ / $\hat{c}$	预测质量 / 预测成本	上方的小帽子表示估计值，并非已知真实结果
λ	lambda，成本权重	把成本换成可与质量比较的惩罚；数值越大越节省
τ	tau，方案得分	用于比较方案，越高越优；并不是成功概率
S / H	候选模型集合 / 已调用集合	候选方案必须包含已调用模型，即 $H \subseteq S$

符号	怎么读	实际含义
j / λ_j	当前阶段 / 该阶段的权重	调用后进入下一阶段，可以使用不同成本权重
B / γ	平均成本预算 / 混合概率	本类 γ 是并列最优时选择较贵方案的概率

“超模型”就是一组候选模型

例如 $S = \{船端，岸端\}$：方案成本是两个模型成本之和，方案质量是“从两份答案中选最好”的预测价值。当前轮只调用其中一个尚未运行的模型，拿到回答后重新决策。

集合成本 = 各模型预测成本相加

集合质量 = “取最高质量”之后，再求平均

集合得分 = 集合质量 − 当前阶段成本权重 × 集合成本

为什么“先取最大值，再平均”顺序不能交换？

下面是解释计算顺序的假想例子，两个情形各占一半：

假想情形	模型 A	模型 B	该情形的最大值
情形 1	0.90	0.50	0.90
情形 2	0.50	0.90	0.90

先取最大值，再平均：$(0.90 + 0.90) \div 2 = \mathbf{0.90}$。
先对每个模型平均，再取最大值：$\max(0.70, 0.70) = \mathbf{0.70}$。

> 这解释的是预测分布中的互补价值，不表示在线可以偷看真实质量。实际输出仍由已返回答案的质量预测决定；本地均值适配器没有协方差，集合质量退化为预测均值中的最大值。

假设并列最优的便宜方案平均成本为 1，较贵方案为 3，目标平均预算 B = 1.5。选择较贵方案的概率 γ = (1.5 − 1) ÷ (3 − 1) = 0.25。

这样混合的平均成本为 0.75 × 1 + 0.25 × 3 = 1.5。clip(..., 0, 1) 表示把计算结果限制在 0 到 1 之间。这个解释要求两个成本不同；相同时不能直接做除法，代码另行处理。

它用于并列最优方案的预算混合，不是任意随机调用模型，也不是每条请求的硬预算保证。

把公式真正算一遍

船端 → 岸端 → 停止

以下沿用已运行的合成教学轨迹：λ = 0.5，γ = 0，无协方差；船／岸／云预测成本为 0.10／0.40／1.00。没有调用真实模型。

第 0 轮 · 没有回答

先调用船端

船端：0.70 − 0.5 × 0.10 = **0.65**
岸端：0.85 − 0.5 × 0.40 = **0.65**
云端：0.90 − 0.5 × 1.00 = **0.40**

船端与岸端并列；γ = 0 时选择便宜方案，所以先调用船端。包含多个模型的集合也没有更高得分。

第 1 轮 · 船端已回答

新反馈让船端质量预测降至 0.40

候选集合	代入公式	得分
仅船端（停止）	0.40 − 0.5 × 0.10	0.35

候选集合	代入公式	得分
船端 + 岸端	**0.85 − 0.5 × (0.10 + 0.40)**	**0.60 ← 最优**
船端 + 云端	0.90 − 0.5 × (0.10 + 1.00)	0.35
船端 + 岸端 + 云端	0.90 − 0.5 × 1.50	0.15

最优集合含有尚未调用的岸端，于是下一步调用岸端。船端已花成本仍保留在集合中。

第 2 轮 • 船端、岸端已回答

新增云端的收益不足以抵消成本

停止：0.86 − 0.5 × 0.50 = **0.61**
加云端：0.90 − 0.5 × 1.50 = **0.15**

继续调用的得分更低，因此停止。随后 `select_answer()` 在已返回答案中选择岸端答案。

> “停止调用”与“选择已有答案”是两个步骤。当前固定均值回放中六条任务都直接调用云端，不能将此教学轨迹当成该回放的实验结果。

完整内容 • 逐章对照源码

原报告全文阅读版

以下保留原报告的全部章节、表格、代码与验证边界。公式已转换为浏览器可直接显示的符号，流程图已重绘；源码链接打开对应文件，原行号仍可在 Markdown 中查阅。

Cascade Routing：论文与本地代码框架解读

日期：2026-10-12。范围：阅读原 Cascade Routing 论文及本地算法、估计器、实验入口、船岸云适配、统一执行与采集评分代码，并执行已有测试和小规模合成演示。未修改算法源码，未调用真实模型，未运行大型实验。

1. 先看清仓库的两层职责

本地项目包含原论文的通用算法库，以及后来增加的船岸云研究工程框架。理解它的关键是分清：**估计器预测各模型的质量与成本，选择算法决定下一步调用或停止，执行器负责实际动作和记账，评估器在终止后判定服务是否成功。**

层	主要文件	解决的问题
通用接口	`base_algorithm.py`、`base_computer.py`	统一选择算法和估计器的调用形式
原论文决策核心	`router.py`、`baseline_cascader.py`、`cascade_router.py`	一次路由、阈值级联、超模型级联路由
质量与成本估计	`quality_computer.py`、`cost_computer.py`、任务专用估计器	根据问题和已返回答案预测质量、成本及不确定性
超模型统计与参数搜索	`statistics.py`、`lambda_strategy.py`	估计集合最优答案质量，选择质量—成本折中参数
原实验组织	`scripts/`、`utils.py`、`notebooks/`	数据转换、拟合、缓存回放、预算扫描、曲线汇总
本地数据与状态接口	`schema.py`、`replay_data.py`	定义在线可见状态与独立评估数据，检查来源分组和缺失值
船岸云策略适配	`cascade_adapter.py`、`policies.py`	将原核心接入受限动作协议及统一执行器
执行与终局评价	`executor.py`、`metrics.py`	动作守卫、时间费用累计、失败并集、未知结果

层	主要文件	解决的问题
小规模离线入口	`replay.py`	拟合开发均值，运行五种策略，输出轨迹与汇总
单条采集和评分	`single_task.py`、`live_backend.py`、`http_once_worker.py`	单次 HTTP、持久化尝试账本、双人评分与独立评价

这里的文件名都位于 `src/selection/`，特殊目录除外。角色为 ship／shore／cloud 的模型名称，不等于已经配置了相应 Qwen 权重或真实部署设备。

2. 原论文的核心：在“模型集合”之间做路由

论文为 Jasper Dekoninck、Maximilian Baader、Martin Vechev 的 *A Unified Approach to Routing and Cascading for LLMs*，ICML 2025，PMLR 267:12987–13010。官方页面。已将官方 PDF 下载到 本目录，并提取为 文本。本次重点阅读 § §2–4 的决策定义、Algorithm 1–3、§5 和附录 D 的估计器设计，并与代码对应；不声称完成附录所有证明的独立数学验证。

2.1 Routing：一次选一个模型

对输入问题 x，估计各模型质量 $\hat{q}_i(x)$ 与成本 $\hat{c}_i(x)$，比较：

$$\tau_i(x) = \hat{q}_i(x) - \lambda \cdot \hat{c}_i(x)$$

`Router._predict_model()` 选择该值最大的模型。λ 越大，越重视成本。论文约束的是请求分布上的**平均成本预算**，不是每请求必不超预算。对应：论文 §2、Algorithm 1；router.py。

2.2 Cascading：固定顺序中的停止或继续

普通阈值级联依次运行模型，在当前答案的质量预测足够高时停止。`BaselineCascader` 对应这种 thresholding baseline：第一步固定第一个模型，后续比较当前模型质量与 `1 - lambda_`，最终选择已运行序列中最后一个模型。

论文还提出更强的固定顺序级联：比较继续到不同后续位置的集合价值，而不只看当前答案门槛。它在代码中由 `CascadeRouter(cascade=True)` 实现，枚举固定前缀并强制紧邻继续。两者都属于 cascading，但决策机制不同。对应：论文 §3、Algorithm 2；baseline_cascader.py、cascade_router.py。

2.3 Cascade Routing：允许初始选模、跳过模型和答案后再选

原方法把一组候选模型称为 supermodel，可译为“超模型”。**它是一个模型集合及其答案选择规则，不是把权重合并成新模型，也不做答案文本融合。**

对集合 S，论文估计：

$$\hat{q}_S = E[\max_{i \in S} \tilde{q}_i]$$
$$\hat{c}_S = \sum_{i \in S} \hat{c}_i$$
$$\tau_S = \hat{q}_S - \lambda_j \cdot \hat{c}_S$$

$\tilde{q}$ 表示估计不确定性下的质量量。当前已经调用的模型集合为 H，则比较的集合须包含 H。如果最优集合没有新增模型，就停止；如果包含尚未运行的模型，就执行一个新增模型，再用实际回答更新估计。

三个模型初始最多有七个非空集合：`{ship}`、`{shore}`、`{cloud}`、`{ship,shore}`、`{ship,cloud}`、`{shore,cloud}`、`{ship,shore,cloud}`。已调用船端后，只比较包含船端的集合。开启向上顺序约束后，一旦跳过某低层模型，之后不能再回头调用它。

对应：论文 §4、Algorithm 3；cascade_router.py。理论上的一般框架允许更灵活的模型序列，论文也给出先运行一个模型、再运行另一个较小模型的例子；本地船岸云协议明确限制严格向上，两者的动作空间不同。

原文的最优性结论针对其定义的估计目标、成本约束和相应假设，不意味着预测器一定正确、真实服务绝对最优或具有服务失败概率保证。

3. 原代码的接口与数据流

3.1 三个数据数组

以 N 条问题、M 个模型为例：

```
questions       # 长度 N，元素可为字符串或任务专用表示
model_answers   # N × M，未调用位置为 None
qualities       # N × M，估计器输出各模型的预测质量
costs           # N × M，估计器输出各模型的预测成本
sigma           # 每问题一个 M × M 协方差矩阵，或 None
```

`model_answers` 是最重要的信息边界。例如 `[ship_answer, None, None]` 只暴露船端回答，岸端和云端仍不可见。已有回答既用于评价该答案，也可以为未调用模型的预测提供反馈。

这些不是原始质量标签矩阵。训练或评估使用的 `ground_truth_qualities` 与在线预测值分开。原实验中的特殊 GroundTruth 估计器是噪声诊断设定，不应直接迁入真实路由器作为可观测输入。

3.2 两类 fit 的职责不同

```
quality_computer.fit(estimator_questions, all_answers, quality_labels)
cost_computer.fit(estimator_questions, all_answers, observed_costs)

router = CascadeRouter(quality_computer, cost_computer, models,
                       max_expected_cost=target_budget)
router.fit(tuning_questions, tuning_answers,
           ground_truth_qualities=tuning_labels,
           ground_truth_costs=tuning_costs)
```

前两次 `fit` 学习预测器；`CascadeRouter.fit()` 调整各阶段 λ_j 和并列最优方案的混合参数 γ。**路由器的 fit 不会自动替你训练质量估计器。** `trigger_training(True)` 主要改变估计器的缓存／运行模式，也不等价于拟合预测器。utils.py、cascade_router.py。

原实验一般把训练数据前段用于估计器，后段用于策略参数搜索；RouterBench 的 GroundTruth 噪声实验另有特殊路径。这个“参数拟合”不应与船岸云论文中的独立失败风险校准混为一谈。

3.3 在线循环由调用方完成

```
answers = [None] * len(models)
while True:
    next_model = router.predict([question], [answers])[0]
    if next_model is None:
        break
    answers[models.index(next_model)] = invoke(next_model, question)

chosen_model = router.select_answer([question], [answers])[0]
chosen_answer = answers[models.index(chosen_model)]
```

`predict()` 返回的是下一模型名称或 `None`，本身不发 LLM 请求。`select_answer()` 返回最终答案所属的模型名称，调用方再取真实回答。代码不是一个调用 `CascadeRouter` 就自动管理全部模型服务的端到端应用。

原实验 utils.prediction() 实现上述循环，但 `invoke` 实际是从缓存 `actual_answers[i][model_index]` 取答案；质量标签在完成选答后用于计分。它是离线评估，不是模型现场生成。

4. 深入核心函数：它每轮做了什么

入口 CascadeRouter.predict() 先调用质量和成本估计器，按已返回答案数确定阶段 j，选择当前阶段的 λ_j，再进入 `_predict_model()`。

该函数的主要逻辑是：

1. 收集非 `None` 的已运行模型，作为所有候选集合的共同基础。
2. 用 `models_to_evaluate` 逐轮扩展未运行模型组合。
3. 对每个集合调用 `predict_supermodels()`，获得集合质量估计，再累加集合成本。
4. 计算 `quality_supermodel - lambda_ * cost_supermodel`，保留最优方案及并列项。
5. 若开启 `do_speedup`，按负边际收益检查剪掉部分扩展；`max_depth` 和 `top_k_keep` 限制搜索规模。
6. 在并列最优集合中，依照 γ 选择便宜或昂贵的方案。该随机化用于预算边界，不是随机挑选任意模型。
7. 最优集合没有新模型则返回 `None`；否则返回其中一个待运行模型，本轮只执行它。

在 `CascadeRouter` 内，γ 是并列时选择较昂贵方案的概率，其拟合形式为：

$$\gamma = \text{clip}((B - C_{\text{cheap}}) / (C_{\text{expensive}} - C_{\text{cheap}}), 0, 1)$$

论文 Algorithm 1 对混合权重使用的方向与这个类不完全相同，不能不看定义就直接照搬变量含义；应以各实现中的分支和公式对应解释。

已有模型的预测成本也出现在候选集合总成本中。在同一轮、同一历史下，它们属于候选间共同的已付出部分；重要区别来自新增模型成本。实际请求记账仍需由调用方或执行器保存，预测成本不能覆盖真实已发生费用。

4.1 不确定性为何重要

BaseQualityComputer.predict_supermodels() 调用 compute_expected_max()：

- 没有协方差时，退化为模型预测均值中的最大值。
- 有协方差时，代码使用 Sobol 样本、正态变换，以及独立或相关质量采样，近似估计最大值的期望。
- 同一集合的模型预测存在相关性时，多个模型的互补价值与完全独立时不同。

$E[\max Q_i]$ 通常不等于 $\max E[Q_i]$。如果把不确定性全部设为 `None`，集合价值的表达能力会明显改变。这是理解本地均值适配较弱的重要原因。

4.2 停止与最终选答是两件事

`predict()` 解决“是否值得继续花钱”；select_answer() 解决“已有的哪个回答更好”。后者只对实际返回的答案重新预测质量，选择预测分最高者，空历史返回 `None`。

因此，最后一个被调用的模型不必是最终输出模型。`BaselineCascader` 则采用最后一个已运行模型，两种选答规则不可混用。

5. 估计器层：通用算法怎样理解不同任务

原选择核心不限定答案必须是普通字符串。各任务把输入和输出转成自己的表示，再实现相同 `predict()` 接口。

任务类型	主要文件	已读到的特征／机制
分类／选择题	`classification.py`	选项概率、熵、最高概率、模型答案一致性、JS 散度、附加问题特征；默认逻辑回归
自由生成	`open_form.py`	继承分类估计框架，改造问题、置信度和答案一致性特征；成本估计结合 token 费率及输出长度
代码／数学	`code_math.py`	任务来源、难度等问题特征，以及解析答案后的反馈；成本回归
SWE-Bench	`swebench.py`	问题与仓库特征、执行反馈／测试信息；时延成本估计
噪声诊断	`quality_computer.py`、`cost_computer.py` 的 GroundTruth 类	在真实值上加噪声以分析估计质量的影响，仅对应明确的实验设定

ClassificationQualityComputer.prepare_data() 将完整训练输出按“哪些模型已运行”的不同子集遮蔽，再为目标模型生成学习样本；`fit()` 按目标模型和可见模型子集分别训练预测器。`predict()` 根据当前非空答案集合选择相应预测器及协方差。

这样，初始没有回答时是 ex-ante 预测；调用后可利用回答反馈进行 post-hoc 预测。**级联的价值依赖调用后是否获得足够有用的新信息。** 如果生成后预测完全不变，系统虽然还能继续调用，决策却缺乏基于答案内容调整的依据。

原 SWE-Bench 部分使用评估测试信息作为良好 post-hoc 估计场景，RouterBench 部分使用带噪声的真实质量信号。本地研究若使用这些机制，必须明确哪些信息部署时真的能获得；不能把受控诊断结果直接当作真实船舰服务能力。

6. 用一条实际运行的教学轨迹理解它

本次新增 trace_native_router.py，只构造合成质量预测和合成返回文本，直接调用原 `CascadeRouter`，不开启 LLM 推理。参数手动设定 $\lambda = 0.5$、$\gamma = 0$、无协方差，用于展示控制流程；它不是拟合模型或论文实验。

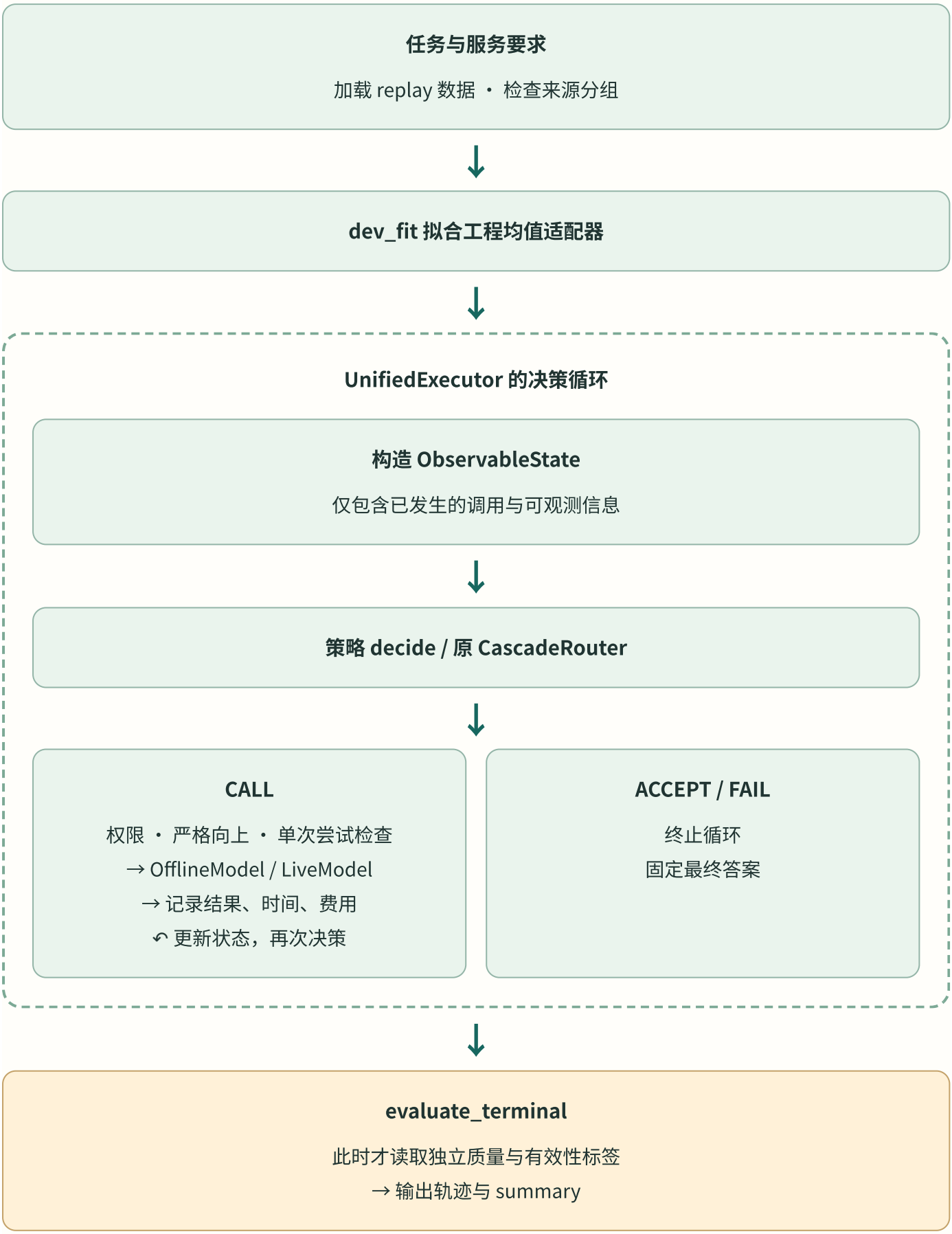
轮次	已返回答案	预测质量：船／岸／云	下一动作
0	全空	0.70／0.85／0.90	调船端
1	船端回答	0.40／0.85／0.90	调岸端
2	船端、岸端回答	0.40／0.86／0.90	停止；最终选岸端

模型预测成本为 0.10／0.40／1.00。初始船端得分 $0.70\text{-}0.5 \times 0.10 = 0.65$，岸端也为 0.65，并列时当前配置选择便宜集合，先运行船端。

船端返回后其质量预测降至 0.40，保留船端并增加岸端的集合得分为 $0.85\text{-}0.5 \times (0.10+0.40) = 0.60$，高于只保留船端的 0.35，因此继续运行岸端。岸端返回后，已有集合的得分为 $0.86\text{-}0.5 \times 0.50 = 0.61$，增加云端后为 $0.90\text{-}0.5 \times 1.50 = 0.15$，所以停止。

原始运行结果见 native_trace.json。这里特意展示“调用后改变预测”，与当前固定均值回放形成对照。

7. 船岸云本地框架：谁负责执行，谁能看到什么



此图表示本地接口逻辑。现有 `replay` 入口使用 `OfflineModel`；`single_task.collect` 使用 `LiveModel` 进行三个独立固定模型请求，并未把图中的动态路由直接接入该采集入口。

7.1 状态与隐藏标签分开

schema.py 的 `ObservableState` 含原始问题、固定证据、可调用模型、已尝试与成功／失败／无效回答、剩余时间预算、预测质量及当前环境。

UnifiedExecutor._make_state() 从已经发生的调用记录构造状态；真实质量、未调用答案不会复制进去。即使回放进程拥有所有缓存输出，也只有执行到相应模型时才暴露该回答。最终选择器也只能从已调用且在线检查成功的回答中选择。

`ModelStatus.SUCCESS` 表示输出通过在线有效性检查，不等于质量已经达到门槛。`FAILED` 表示尝试失败，`INVALID` 表示调用发生但输出无效，`UNATTEMPTED` 表示未调用。这些状态影响重试限制和费用账本，必须区分。

7.2 执行器统一管理动作与资源

UnifiedExecutor.execute() 每轮构造状态、运行策略、计入控制开销、检查动作，再调用模型。失败和无效结果仍加入尝试历史并保留资源。发生资源未知时停止进一步花费，未知值不被写成零。

`_validate_call()` 强制权限、未尝试和严格向上顺序。模型接口只读取同一原始 `question` 与 `evidence`，不接收前模型答案。

回放模式用输入的延迟和费用累计；真实后端必须使用 `execution_timing="wall_clock"` 与 `decision_timing="measured"`，用完整请求的单调时钟差计总时间，分项只作诊断，避免重复叠加。

但当前执行器尚不包含船岸云论文完整的预测上界动作筛选、保守停止门和多目标规划机制。代码中的 ACCEPT 表示终止并返回已选答案，**不自动表示真实服务达标**，须由终局评估判断。

7.3 终局评估是失败并集，缺失值保留未知

metrics.evaluate_terminal() 在选答已固定后读取标签，检查质量、时间、预算和有效答案；不会重新调用模型或根据标签改选答案。

如果已知任一条件失败，结果就是失败；如果尚无已知失败但必要信息缺失，则结果不确定。`aggregate()` 以全部请求为分母，存在未知结局时不伪造完整失败率，而是报告确定失败下界与可能失败上界。这些上下界是缺失信息范围，不是统计置信区间。

8. 当前 CascadeRouterAdapter 到底实现了什么

cascade_adapter.py 保留原 `CascadeRouter.fit/predict/select_answer` 调用链，外面增加严格向上、权限、失败占次数等适配。失败或无效尝试用非空哨兵表示，并赋极低预测质量，防止原代码把它当成尚未运行或最终好答案。

但估计器是 `DevelopmentMeanQuality` 和 `DevelopmentMeanCost`：只在 `dev_fit` 上计算每模型均值，在线对所有任务基本返回同一组预测；质量没有协方差，λ 固定为零，不做参数搜索，只按拟合数据计算 γ。适配器也显式拒绝其他估计器类型，尚不能直接换入任务专用估计器便宣称适配完成。

本次运行的配置给出：

```
quality_means = [0.55, 0.78, 0.93]
cost_means    = [1.00, 2.00, 3.00]
lambdas       = [0, 0, 0]
gamma         = 1/3
```

六条 `dev_eval` 任务都直接调用云端；两条成功、另有无效输出、资源超限以及标签／资源未知等诊断结果。这说明原核心已接通执行器和评估，但示例未体现任务感知或回答质量感知的动态升级。它不能作为 v5 论文中强序贯基线或完整方法的性能复现。

`fixed_cascade_fallback` 同样只耗尽固定顺序、返回最后有效答案，是明确命名的替代规则；它尚不是论文中带训练评分器的固定阈值级联。

9. 单条采集与评分链的定位

入口 single_task.py 的流程为：

```
validate 本地校验
  → collect 分别执行船、岸、云三个固定单模型请求
  → 保存原始输出与匿名评分卡
  → 双人评分及分歧裁定
  → evaluate 合并评估标签并重新计算终局指标
```

`collect` 是采集三模型数据，不是一条动态级联轨迹。每次独立调用均复用 `UnifiedExecutor`，以保证接口和记账一致；三个独立延迟不能直接拼成真实序贯网络轨迹。

真实后端的调用链为：

```
UnifiedExecutor
  → LiveModel.call_with_context
  → AttemptLedger.reserve（SQLite 持久化唯一尝试）
  → SingleHTTPTransport.send
  → http_once_worker.send_once（单次标准库 POST）
  → 解析、检查、记账并保存结果
```

PENDING 和已完成尝试都阻止同任务、同模型自动重发。worker 被本地超时终止，不意味着远端生成或费用已经取消。当前费用仅按 provider token fee 推导，不是 v5 的费用—传输归一化开销。

原 `APIQuery` 则是 LangChain／供应商 SDK 适配层，有多层重试路径；本地新后端避免走这条链。当前环境 `APIQuery` 的可选导入是否成功，应直接检查具体依赖，不能因为 `import selection` 成功就认为所有子模块都可用，因为 `__init__.py` 会捕获部分导入异常。

10. 与 papers/v5 方法的对应关系

v5 所需机制	当前本地代码状态
原始问题和证据固定、每模型至多尝试一次、向上可跳级	执行器、状态与适配器已实现相应工程约束
实际时间／费用账本、失败调用记账、标签与决策分离	已有实现与诊断测试
原 Cascade Routing 的受限接入	核心接通，估计器仍为开发均值变体
MiniLM＋PCA＋梯度提升质量与达标概率预测	当前这条本地船岸云路径尚未实现
条件升级收益模型与收益增强近邻	尚未实现
答案—资源联合轨迹采样	尚未实现
四维非支配前沿、压缩、多步滚动规划	尚未实现；原超模型搜索不是该规划器
共享上述预测器的单步贪心对照	尚未完成；原 `greedy=True` 不能直接等同
完整冻结控制器的独立风险校准	尚未实现；回放输出明确 `risk_guarantee=false`
v5 的领域数据、真实完整实验与可复现部署记录	现有合成演示和源码不能验证这些结果

原论文中的正态质量采样与 v5 中的历史结果—资源联合采样属于不同机制。 原算法搜索模型集合的期望最大质量和成本；v5 规划器还要对未来可观测状态、资源消耗及失败概率作递推。复用执行框架有意义，但不能简单把 `CascadeRouter` 改名为完整规划器。

另有一处文档陈旧信息：`replay.py` 生成的 `manifest.not_implemented` 仍包含 `live_no_retry_backend`，但仓库实际上已有 `live_backend.py`、worker 和测试。它反映回放清单未随新后端同步，不能据此判断整个仓库没有该后端。本次仅记录这一问题，未修改运行源码。

11. 现有验证与解释边界

实际运行结果如下，命令和退出状态见 commands.json：

操作	结果
沙箱内现有 pytest 全套	74 passed、7 errors；7 项在本机 HTTP fixture 建立 socket 时被权限限制
允许本机 socket 后全套 pytest	81 passed，退出 0，4.65 秒
`selection.replay --validate-only`	退出 0；10 任务、30 调用记录、30 标签记录
小规模离线回放	退出 0；4 个拟合任务、6 个评价任务、5 策略，共 30 条评价轨迹
原核心教学轨迹	退出 0；3 次决策，2 个合成答案槽填充，最终岸端；无真实生成

测试和回放日志保存在本目录；tests_full.xml 为最终测试统计。81 项主要验证本地接口、失败资源处理、数据隔离、单次尝试、模拟 HTTP 和评分闭环，并不覆盖原论文的全部任务估计器、预算搜索、无序选择与所有消融配置。因此测试通过只证明这些已测试的软件行为，不证明论文性能或风险达标。

阅读中还注意到原核心无序选择分支使用 `best_models[model]`，而本函数通常只维护 `best_models['all']`；该分支值得单独审计。当前船岸云路径固定 `force_order=True`，本次未修改也未验收一般无序配置。不要把受限路径的测试通过推广到所有构造参数。

12. 推荐的源码学习顺序

1. base_algorithm.py 与 base_computer.py：先看 fit／predict／select_answer 的职责。
2. router.py：理解最简单的质量减成本打分。

3. quality_computer.py 与 statistics.py：理解超模型和不确定性。
4. cascade_router.py：按 predict → _predict_model → select_answer → fit 顺序读。
5. classification.py：理解生成前后信息如何进入预测器。
6. utils.py：理解原实验的估计器拟合、预算搜索和缓存评估。
7. executor.py 与 cascade_adapter.py：理解船岸云如何接入原算法。
8. replay.py、metrics.py：结合本目录轨迹查看决策与事后评价的分界。
9. single_task.py 与 live_backend.py：最后看真实调用接口和数据采集。

这条学习顺序先掌握模型选择的数学逻辑，再读任务特征和执行约束，可避免一开始陷入各数据集脚本、API 封装或旧实验配置。

由 render-html skill 的 Markdown 解析器与本目录 build_reader.py 生成。新增中文说明和教学算例用于辅助理解；原报告的验证结果与实现边界保留。单文件 HTML，无 CDN、无脚本、无需联网。